

DAY 15: lambda function

→ lambda function is small anonymous function

→ lambda can take 'n'-number of arguments, but only with one expression

→ The function is defined by using lambda keywords

Syntax → lambda arguments : expression

Eg:

```
add = lambda a,b: a+b  
print(add(10,9))
```

```
great =lambda a,b: a if a>b else b  
print(great(10,20))
```

filter ()

→ filter () function will perform only on selected elements of iterables

syntax → filter(lambda arguments: expression ,iterable)

Eg:

```
nums =[1,2,3,4,5]  
data = list(filter(lambda a: a%2==0, nums))  
print(data)
```

map ()

→ map () function will perform on every elements of the iterables

syntax→ map(lambda arguments: expression, iterable)

Eg:

```
nums=[1,2,3,4,5]  
get =map(lambda a: a**3,nums)  
print(list(get))
```

reduce ()

→reduce () function repeatedly applies a function to the elements and reduces them to one final value.

→ It is available in the functools module.

syntax→ reduce(lambda arguments: expression,iterable)

Eg:

```
from functools import reduce  
nums =[1,2,3,4,5]  
data=reduce(lambda a,b:a*b,nums)  
print(data)
```